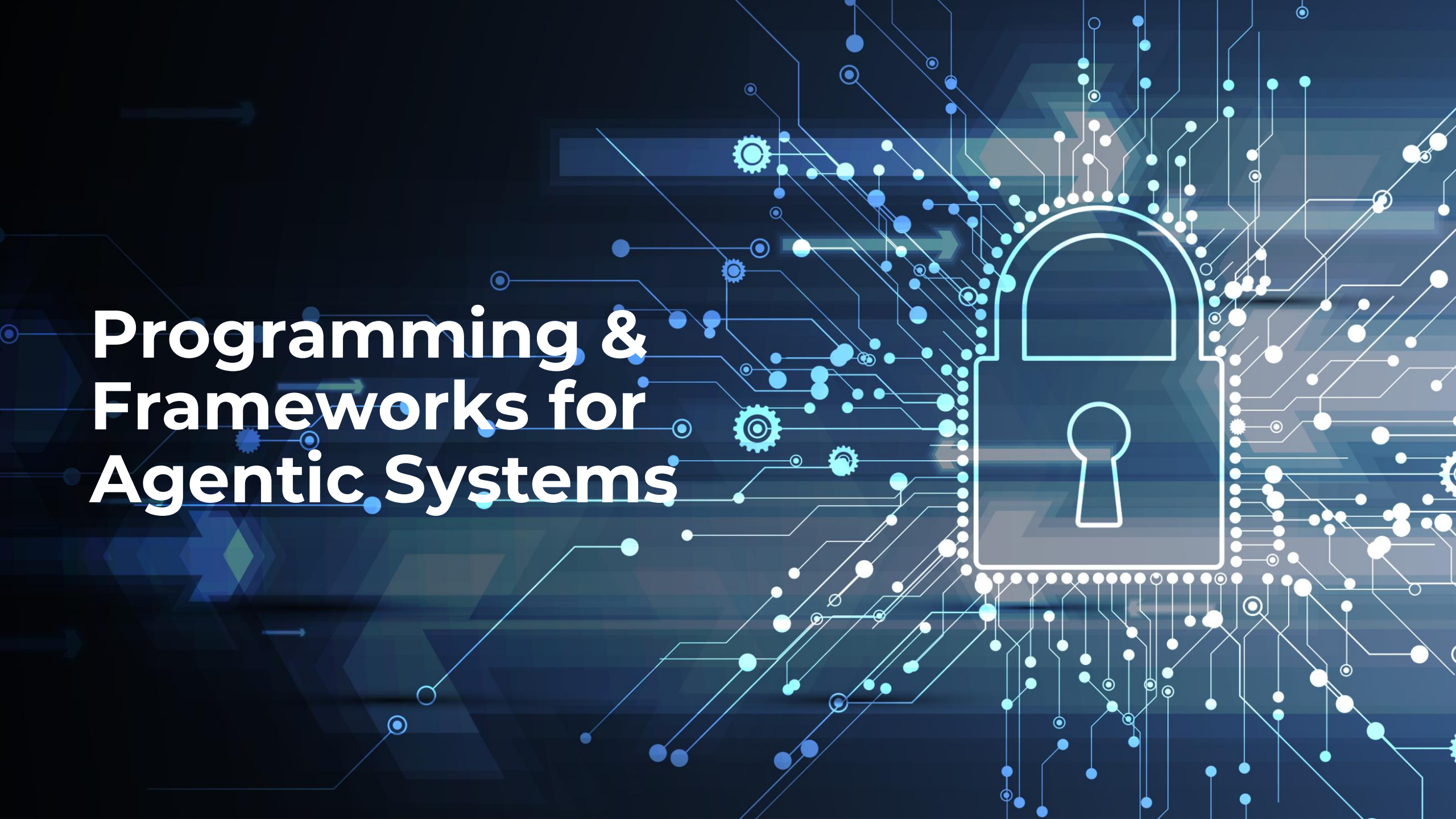
IITM PRAVARTAK
CATALYSING INNOVATION

Professional Certificate Programme in Agentic AI and Applications

Programming & Frameworks for Agentic Systems



Revisit the Learning So Far

Let’s take a quick look at the key ideas and concepts you have explored in your learning journey.

- What Agentic AI is and how it differs from traditional automation
- How Agentic AI emerged and why it is becoming increasingly influential
- How the agent life cycle works, from perception to action and learning
- How autonomy levels define an agent’s maturity and decision-making capabilities
- How rule-based systems compare with Agentic AI in flexibility and adaptation
- A live demonstration showing how an agent moves from a goal to an actionable plan

Topics Covered

- Understanding the AI Ecosystem: Agents, Functions and APIs
- The Agent Life Cycle: From Goals to Growth
- Agentic Frameworks: LangChain, Autogen, CrewAI etc.
- AI Agent Frameworks: Core Principles for Tool Integration
- Extending AI Agents with External APIs

Understanding the AI Ecosystem: Agents, Functions and APIs

Understanding the AI Ecosystem

- Distinguish between functions, APIs, and agents in modern software development
- Understand how these components interact to enable AI-powered systems

Functions v. APIs: The Building Blocks

Functions

Self-contained code blocks that process input and return output

- Single-purpose, predictable
- Operate within the same codebase
- Example: `add(5, 3)` returns 8

Analogy: Like specialised kitchen appliances performing one task efficiently

APIs

Gateways for communication between different software systems

- Access external functionality or data across network boundaries
- Example: Weather API providing forecast data

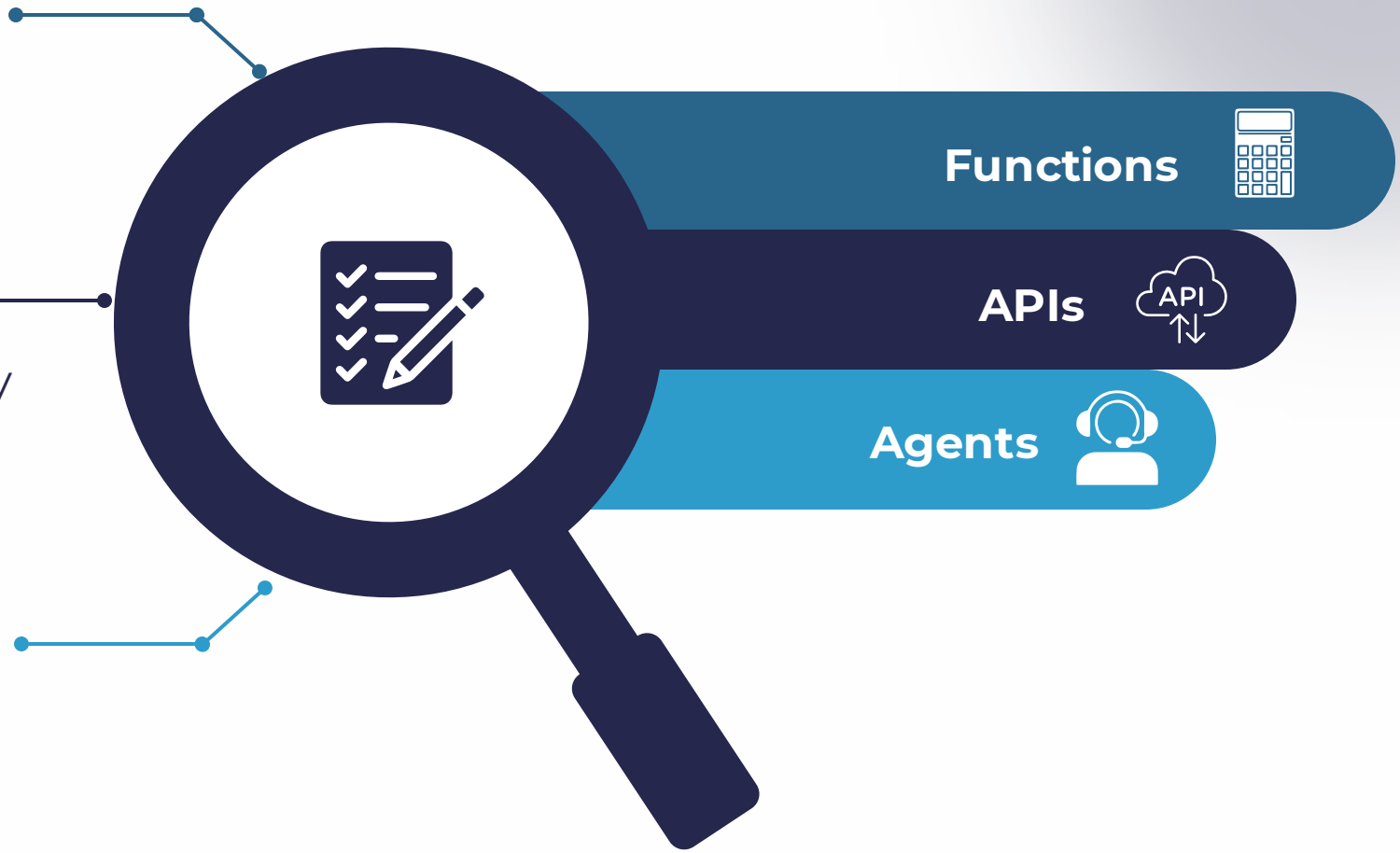
Analogy: Like restaurant waiters—taking requests and returning responses from another system

Key Takeaways for Technical Professionals

The atoms of computation that perform specific tasks within a system

The bridges that connect systems, enabling access to external data and functionality

The orchestrators that reason about goals, strategically using functions and APIs to achieve outcomes



As AI evolves, mastering these distinctions is key—top systems combine functions for logic, APIs for connectivity, and agents for autonomous execution.

Key Takeaways for Technical Professionals

When designing your next system, consider:

Do you need a simple function, an API interface or a full-fledged agent to solve your problem?

Agents: The Intelligent Orchestrators

Agents autonomously select and sequence tools to achieve objectives

Agents plan and adapt to deliver complex outcomes

Agents choose functions and APIs based on context and goals

Goal-driven systems

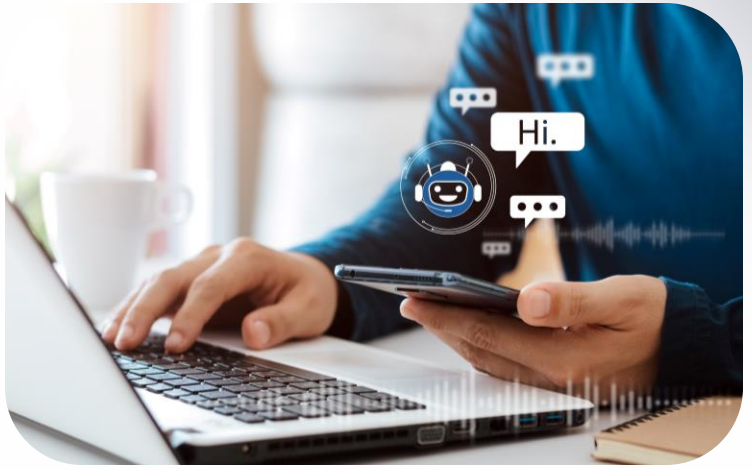
Reasoning capability

Tool utilisation

Agents: The Intelligent Orchestrators

Example

- A travel agent AI searches flights, checks weather, compares prices and books the best option
- All triggered by a simple request like “Plan my vacation to Hawaii”



Analogy

- Like a skilled personal assistant who manages entire projects, not just isolated tasks

The Agent Life Cycle: From Goals to Growth

The Agent Life Cycle: From Goals to Growth

- Understand the full life cycle of intelligent agent systems.
- Explore six key phases that guide agent operation, adaptation and evolution.

Phase 1: Goal Setting

- The agent life cycle begins with clearly defining objectives.
- These goals may originate from:
  - Direct user instructions
  - Environmental triggers or conditions
  - System-level priorities

Clear goals lay the foundation for purposeful and directed agent behaviour.

Phases 2 & 3: Planning and Execution

Planning

Divide objectives into sub-goals and crafting strategies to achieve them

- Prioritising tasks
- Resource allocation
- Contingency planning

Action execution

Execute planned strategies using tools, APIs and systems to complete tasks

- Tool selection and utilisation
- API integration
- System interaction

These phases turn abstract goals into concrete actions, enabling meaningful progress.

Phase 4: Monitoring

Continuous tracking and assessment

Agents monitor progress and stay alert to environmental changes

This critical phase involves:

- **Progress tracking:** Measure advancement toward objectives
- **Environmental scanning** – Detect changes that may affect outcomes
- **Performance analysis**: Evaluate strategy effectiveness

Phase 5: Adaptation

When progress stalls or conditions shift, agents adapt with flexibility and responsiveness.

Detect change

Identify deviations from expected conditions or results

Analyse impact

Assess how changes affect the current plan

Modify strategy

Adjust approach to accommodate new circumstances

Implement

Execute the revised plan

Phase 6: Learning

The final phase enhances future performance through experience, creating a virtuous cycle of increasing effectiveness.

Learning transforms the agent life cycle from a linear process into an upward spiral of continuous improvement.

Key learning mechanisms include:

- Pattern recognition from past experiences
- Strategy effectiveness evaluation
- Knowledge base expansion
- Optimisation of decision-making processes

Agentic Frameworks: LangChain, Autogen, CrewAI etc.

Agentic Frameworks: LangChain, Autogen, CrewAI

- Examine leading frameworks for building AI agent systems.
- Analyse technical strengths and limitations of each framework.
- Identify optimal use cases for development teams.

LangChain: The Established Ecosystem

Open-source Framework for LLM Applications

LangChain offers strong abstractions and seamless integration with databases, APIs and tools

Its memory handling, orchestration, and documentation make it ideal for production use

LangChain: The Established Ecosystem

Strengths

- Rich ecosystem (100+ integrations)
- Strong community support
- Mature visual builder (LangFlow)

Limitations

- Steep learning curve
- Performance overhead
- Rapid evolution impacts stability

Autogen: Multi-Agent Collaboration

Microsoft’s Framework for Agent-Agent Conversation

Core concept

- Autogen enables multi-agent and human interaction through sophisticated dialogue patterns
- It excels at complex reasoning via collaborative exchanges between specialised agents

Key differentiator: Supports native LLM↔LLM conversations with minimal human input

CrewAI: Role-Based Collaboration

Team structure

Organises agents as "crew members" with distinct roles, responsibilities and expertise

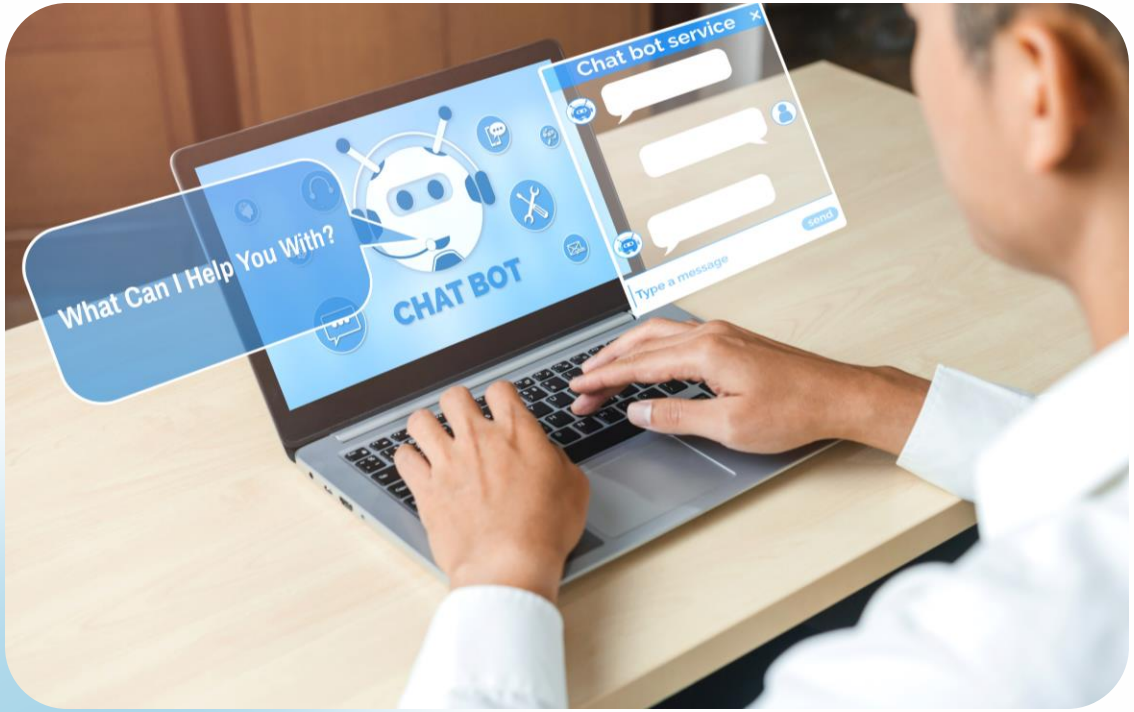
Workflow focus

Enables smooth handoffs between specialised agents in structured - sequences

Lightweight

Offers simpler implementation with minimal overhead compared to more complex frameworks

CrewAI: Role-Based Collaboration



- Models agents after human teams with defined roles, expertise levels and goals
- Excels at structured workflows where tasks are cleanly delegated to specialised agents in coordination

Framework Comparison: When to Use Each

LangChain	Autogen	CrewAI
Production-ready applications requiring many integrations	Complex reasoning requiring multi-step collaboration	Structured workflows with clear role delegation
Document Q&A and retrieval-augmented generation	When agent-agent conversation patterns are central	Content pipelines (research → writing → editing)
Teams with resources for steeper learning curve	Research applications where process visibility matters	Teams seeking faster implementation with less complexity

Framework Comparison: When To Use Each

Framework	Language support	Key strengths	Weaknesses / limits	Best use cases
LangChain	Python, JS/TS	Huge ecosystem, rich integrations, memory & tools	Can be heavy/complex, evolving APIs	Prototyping LLM apps, enterprise pipelines
AutoGen (Microsoft)	Python only	Multi-agent orchestration, conversational agents	Less ecosystem maturity, mostly research-focused	Research, agent collaboration, tool use
CrewAI	Python	Team-based multi-agent coordination (roles/goals)	Young project, smaller community	Workflow automation, agent collaboration
LlamaIndex (GPT Index)	Python, JS	Strong data connectors, retrieval & RAG pipelines	Not designed for multi-agent orchestration	Knowledge-grounded agents, document Q&A

Framework Comparison: When To Use Each

Framework	Language support	Key strengths	Weaknesses / limits	Best use cases
Haystack	Python	Open-source, production-ready for search/RAG	Less multi-agent focus	RAG, search, production NLP systems
Semantic Kernel (Microsoft)	Python, .NET, JS	Skill-based orchestration, enterprise-friendly	Steeper learning curve, MS-centric	Enterprise AI orchestration, C#/MS shops
DSPy (Stanford)	Python	Declarative LLM programming, optimiser-driven	Research-oriented, smaller ecosystem	Fine-tuning prompts, academic/ML workflows
SmolAgents (Hugging Face)	Python	Lightweight agent framework, integrates with Hugging Face hub	Early stage, minimal docs	Quick, small agents, model hub integrations
OpenAI Swarm	Python (experimental)	Focused on multi-agent “swarms” with simple APIs	Very early stage, experimental	Exploratory multi-agent research

Framework v. SDK

Agentic Frameworks (e.g. LangChain, AutoGen, CrewAI)

- Enable high-level orchestration with memory, tools, multi-agent workflows and chaining logic
- Function like application frameworks for building structured AI systems

SDKs (e.g. OpenAI, Google, Anthropic, Cohere)

- Provide low-level API access to models for chat, embeddings and completions
- Lack built-in orchestration such as memory or multi-agent coordination

Framework v. SDK

SDK / Client	Language support	Key strengths	Weaknesses / limits	Agentic relevance
OpenAI SDK	Python, JS/TS	Direct, simple access to GPT (chat, embeddings, tools); strong ecosystem	No native orchestration, memory, or multi-agent features	Use as core LLM engine inside LangChain, CrewAI or custom agent code
Google AI SDK (Vertex AI / Gemini)	Python, JS, Go, Java	Multimodal support (text, image, video, speech); strong enterprise security; integrates with GCP tools	Requires GCP setup & billing; less community content	Good for enterprise-grade agents with multimodal pipelines
Anthropic SDK	Python, JS	Claude models: long context, safe outputs, constitutional AI	Limited ecosystem vs OpenAI/Google	Ideal for safety-conscious agents or long-context tasks

Framework v. SDK

SDK / Client	Language support	Key strengths	Weaknesses / limits	Agentic relevance
Cohere SDK	Python, JS	Powerful embeddings & reranking; focus on retrieval and classification	Not focused on orchestration or planning	Best for RAG agents and knowledge-grounded workflows
Mistral SDK / API	Python, JS (via open APIs)	Lightweight, fast open-weight models; good for local or hybrid agents	Smaller ecosystem; early tooling	Suited for open-source-first agent stacks

Key Takeaways for Technical Decision Makers

Use LangChain for integration-heavy apps, Autogen for complex reasoning and CrewAI for intuitive team workflows

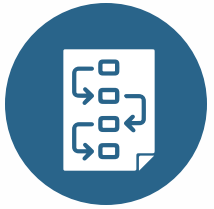
Balance ecosystem maturity against implementation complexity and performance requirements

Choose frameworks based on current needs and long-term roadmap compatibility

Consider your specific use case

Evaluate technical trade-offs

Plan for the future



Key Takeaways for Technical Decision Makers

“The right framework isn't about which has the most features, but which matches your specific agent architecture requirements.”

Prototype across frameworks to gain insights before committing to production architecture.

Contact us: For implementation guidance and architecture reviews tailored to your specific use case.

AI Agent Frameworks: Core Principles for Tool Integration

A technical exploration of how modern agent frameworks manage tools, orchestration and workflows to create reliable AI systems.

Tool Abstraction Fundamentals

Definition

- Defines tools as external resources agents can use, e.g. APIs, databases, calculators, search engines
- Applies tool abstraction to create standard interfaces that shield agents from implementation details

Metadata wrapping

- Packages tools with metadata: name, description, input/output schemas, examples and constraints

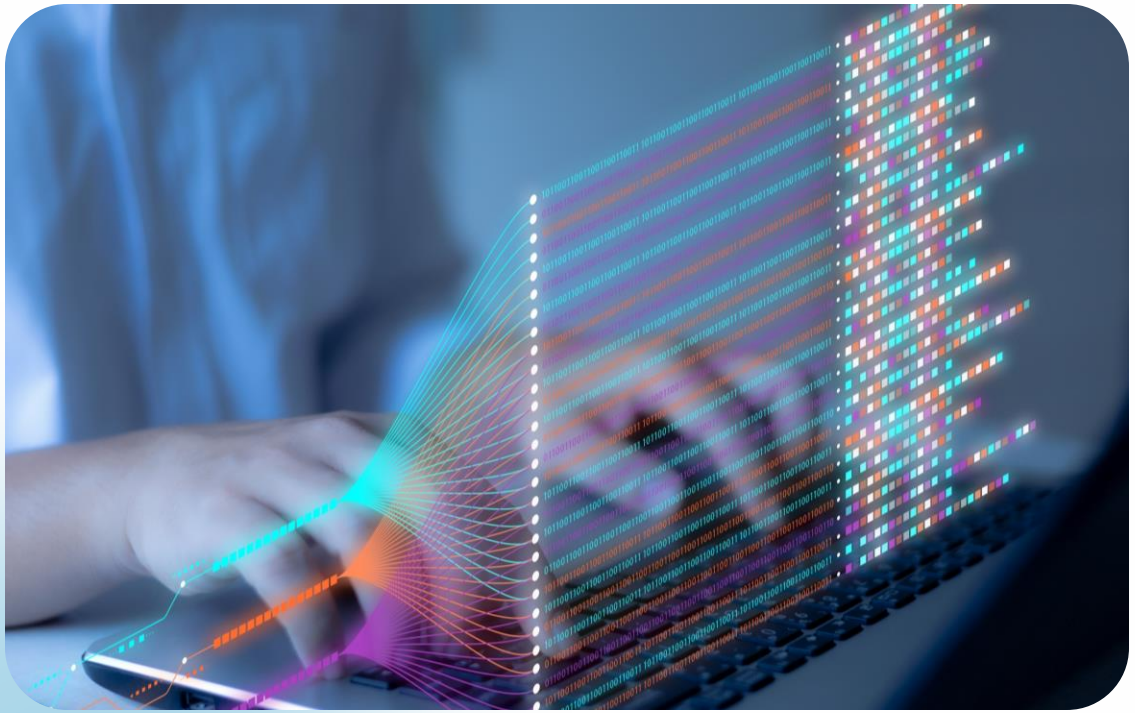
Dynamic selection

- Enables contextual tool choice at runtime, allowing agents to select the right tool for each subtask

Tools simplify interaction by exposing inputs and outputs while hiding internal complexity like smart appliances.

Implementation Example: Tool Definition

Key Components



In frameworks like LangChain, tools are defined as functions with:

- ✓ Descriptive name
- ✓ Documentation string
- ✓ Input/output specifications
- ✓ Error handling

This standard interface allows the agent to reason about when and how to use each tool.

Implementation Example: Tool Definition

```
def weather_tool(location: str, date: str) -> str:    """Get weather forecast for a location.
          Args:        location: City and state/country        date: Date in YYYY-MM-DD
    format            Returns:        Weather forecast as a string    """    # Implementation
    details hidden from agent    ...    return forecast·
```

The agent sees only the interface, not the implementation details of how the weather data is retrieved.

Orchestration: Coordinating Multiple Tools

Sequential	Conditional	Parallel
• Calls tools in order, passing outputs to the next step • Example: Search flights → check hotels → confirm booking	• Selects tools based on context or prior results • Example: If flight price > $500, search alternative dates	• Runs multiple tools simultaneously for independent tasks • Example: Checking weather and hotel options concurrently

Orchestration: Coordinating Multiple Tools

Key Points to Consider

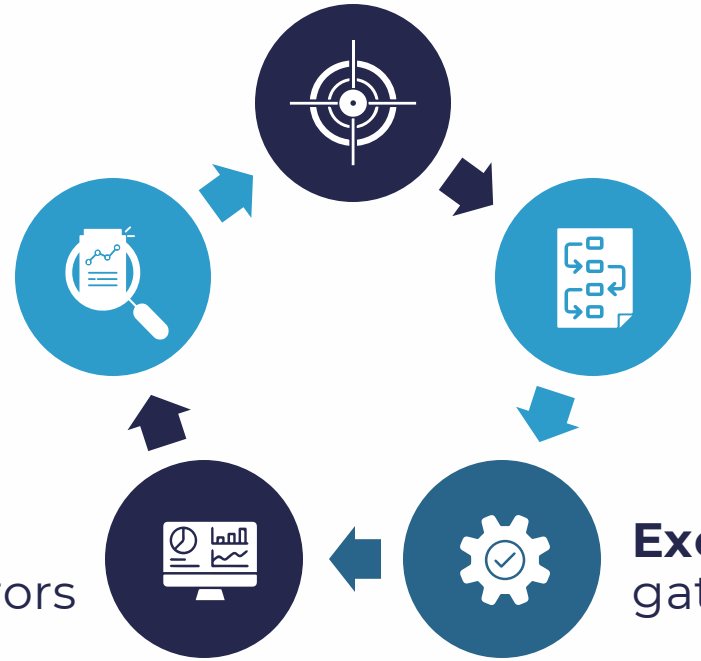
Requires robust reasoning, e.g. chain-of-thought, planners, decision trees, to guide tool use and timing

Coordinates tools like a conductor, ensuring each plays its part to produce a harmonious outcome

Workflow Management Architecture

Critical Workflow Components

Goal definition: Understanding the task objective



Planning: Breaking down into manageable steps

Execution: Calling tools and gathering results

Monitoring: Tracking progress and detecting errors

Reflection: Evaluating results and adapting strategy

Frameworks like AutoGen, LangChain and CrewAI provide primitives to implement these components.

Human-in-the-Loop Integration

Approval checkpoints
Require human confirmation at critical decision points

Guidance interventions
Allow humans to provide context or redirect stalled agents

Result verification
Enable experts to validate outputs before production use

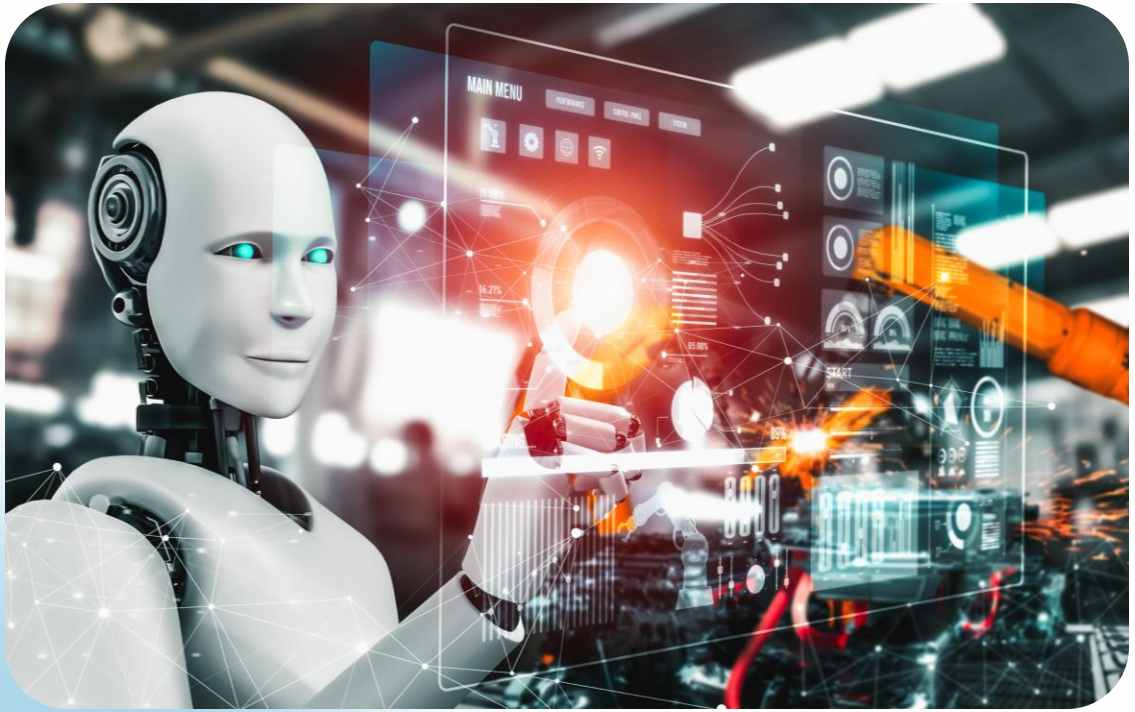
Human-in-the-Loop Integration

> Effective agent systems don't replace humans—they augment them by handling routine tasks while escalating important decisions

Human oversight is vital for high-stakes tasks involving finance, law or sensitive data, where errors carry serious consequences.

Debugging and Observability

Essential Monitoring Capabilities



Robust agent frameworks provide comprehensive observability:

- **Execution traces**: Complete record of tool calls and decisions
- **Thought processes**: Chain-of-thought reasoning for each step
- **Performance metrics**: Success rates, completion times, cost
- **Error patterns**: Common failure modes and recovery attempts

Advanced frameworks offer real-time dashboards to reveal agent workflows, bottlenecks and reasoning failures.

Key Takeaways

Standardises interfaces so agents can use diverse tools without knowing implementation details

Tool abstraction

Coordinates tools effectively—sequentially, conditionally or in parallel—using robust reasoning mechanisms

Orchestration

Structures processes for debugging, monitoring and human oversight to ensure safety and reliability

Workflow management

Choose agent frameworks that offer clear abstractions, flexible orchestration and transparent, human-guided workflows.

Questions? Let's dive deeper into specific implementation approaches for your use case.

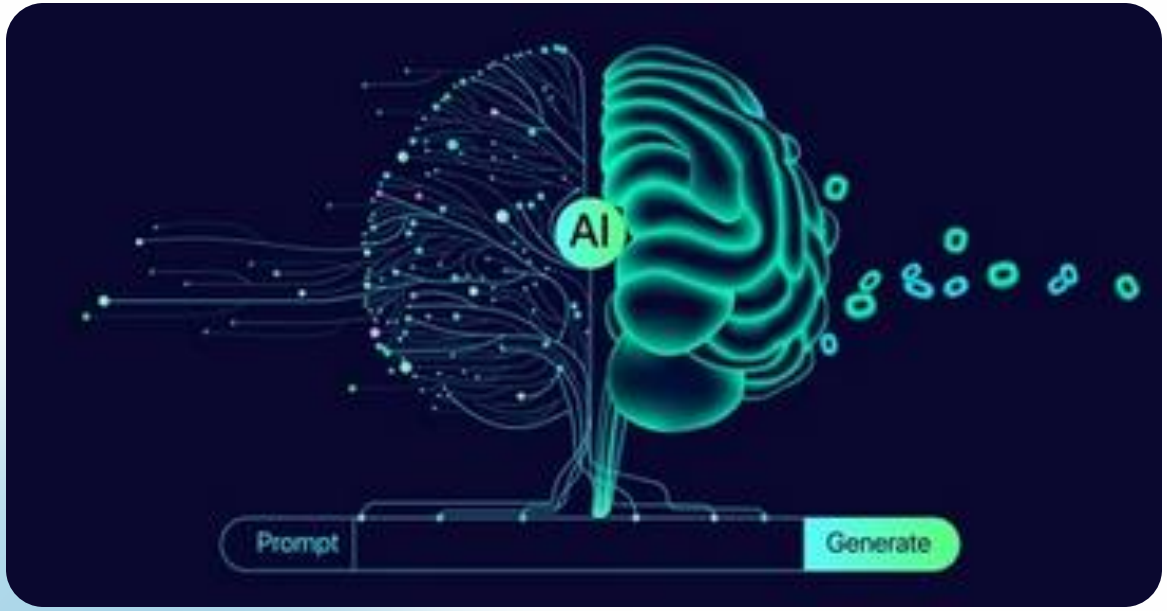
Extending AI Agents with External APIs

A practical guide for developers, AI engineers, and technical product managers on how agents access and leverage external capabilities through APIs.

(Topic 5 of 5)

Why Agents Need External Tools

Modern agents surpass traditional AI by connecting to external tools via APIs, extending beyond pre-trained knowledge.



Limited internal knowledge: Agents rely on static training data and need external connections for real-time information

Need for dynamic actions: Agents must interact beyond core architecture to operate effectively in changing environments

The Agent-API Interaction Flow

User query: User presents a goal or question requiring external data or capabilities

Tool selection: Agent identifies the appropriate API to fulfil the request

API call: Agent constructs and sends the request to the external service

Response processing: Agent parses the returned data and integrates it into its reasoning

Response generation: - Agent produces final output combining reasoning with external data

Real-World Example: Weather Assistant

User query
"What's the weather in Paris? Should I carry an umbrella?“

Agent processing

- Recognises need for current weather data
- Calls Weather API with location parameter
- Receives precipitation and forecast data
- Applies reasoning about umbrella necessity
- Formulates helpful response with justification

Agent response: "It's currently 58°F in Paris with a 70% chance of rain in the next 3 hours. Yes, bringing an umbrella is recommended."

Authentication: Securing API Access

- Simple string tokens that identify and authorise the calling application
- **Example:**

```
X-API-KEY: sk_1a2b3c4d5e6f7g8h9i0j
```

- Delegated access mechanism for user resources via complex authentication
- **Application:** Social media, email and user-data-centric services

- Encoded tokens containing identity and permission claims
- **Advantage:** Stateless verification with expiration controls

API keys

OAuth flows

JWT tokens

Never hardcode credentials in your agent implementation!
Use environment variables, secret managers, or secure vaults.

Authentication Best Practices

Use secret management

Store credentials securely in secret managers like AWS Secrets Manager, HashiCorp Vault or environment variables

Implement token rotation

Refresh OAuth tokens automatically before expiry to maintain access

Apply least privilege

Request only essential permissions needed for agent functionality

API authentication is like a hotel key card—granting limited access and expiring after your stay.

Request/Response Handling

Request construction

- Format endpoint URL correctly
- Include necessary query parameters
- Set appropriate headers (Content-Type, Accept)
- Structure request body according to API specs

Response processing

- Parse returned JSON/XML data structure
- Validate received data matches expected schema
- Extract relevant information for agent reasoning
- Handle pagination for large result sets

```
GET /weather?city=Paris&units=metric
```

```
{"temp": 18.2, "humidity": 65, "rain_prob": 0.7}
```

Request/Response Handling

Error handling

- Detect HTTP error status codes (4xx, 5xx)
- Implement retries with exponential backoff
- Provide graceful fallbacks when APIs are unavailable
- Log failures for debugging and monitoring

Safety Concerns and Mitigations

Key Risks

API access is like handing a child your credit card—set limits, monitor use and require approval.

Rate limiting: Exceeding API quotas can disrupt service availability

Data leakage: Sensitive information may be exposed in prompts or responses

Prompt injection: Malicious users manipulating the agent to make harmful API calls

Cascade failures: API errors causing agent functionality collapse

Unintended consequences: Agent making irreversible changes (deletions, financial transactions)

Implementing Safe API Integration

Rate limit management

Use client-side throttling and queuing to avoid API quota exhaustion

```
if (requestCount > HOURLY_LIMIT * 0.9) {  await delayNextRequest();}
```

Request validation

Sanitise inputs and validate API requests before sending

```
if (!isValidRequest(generatedRequest)) {  return safeErrorResponse();}
```

Human-in-the-loop

Require explicit confirmation for high-risk operations

```
if (isHighRiskAction(request)) {  await requestHumanApproval();}
```

Comprehensive logging

Track all API interactions for audit, debugging and improvement

```
logger.info({ request, response, processingTime});
```

Key Takeaways

Extend capabilities

APIs empower agents by connecting them to external tools, data and actions

Secure access

Use strong authentication and manage credentials to protect sensitive API access

Handle data properly

Structure requests and process responses to ensure reliable agent behaviour

Prioritise safety

Embed safeguards, monitoring and human oversight for responsible API use

Next steps: Explore API frameworks, set up a sandbox, and build thorough testing protocols for agent implementation.

What is a Tool-Using Agent

Beyond "Thinking"
Agents know when and how to use tools like APIs, databases or code to reach a goal

Key advantage
Unlike regular LLMs, tool-using agents delegate factual and computational tasks to specialised systems

Real-world value
This integration turns LLMs into action-capable assistants, not just smart autocompletes

Setting Up Your First Tool-Using Agent

Required Components

LLM wrapper (OpenAi, Anthropic, etc.)

Agent framework (LangChain, AutoGen, etc.)

Tool definitions (functions the agent can call)

Prompt templates (instructions for the agent)

We'll start with a calculator tool that helps our agent solve math problems accurately.

The Power of Tool-Using Agents

LLM-only approach

When asked: "What is 3048 × 1429?":

- I'll calculate 3048 × 1429. 3048 × 1429 = 4,355,592
- **Actual answer: 4,355,592**

Sometimes correct, but unreliable for complex calculations

Tool-using agent

When asked: "What is 3048 × 1429?“:

- I'll use the calculator tool for this. Action: Calculator Action Input: 3048 * 1429 Observation: 4355592
- **Guaranteed accuracy: 4,355,592**

Consistently correct by delegating to specialised tools

Visual Orchestration with LangFlow/Flowise

Visual orchestration tools enable agent creation without coding, using simple drag-and-drop interfaces.

Analogy

"It's like drawing a flowchart, but the boxes are live AI components"

- **LangFlow:** Visual IDE for LangChain components with Python backend
- **Flowise:** Node.js-based visual builder with TypeScript components

Langflow vs. Flowise – Visual Builders for AI Agents

Feature	Langflow	Flowise
Focus	Visual builder for LangChain apps	Low-code editor for LangChain pipelines
Deployment	Export as Python code; dev-friendly	Deploy as APIs; easier for no-code users
Ecosystem	Tighter tie-in with LangChain core	Fast-growing, strong community connectors
Best for	Developers/prototypers needing code export	Citizen developers / teams needing quick apps

Langflow & Flowise

Langflow

- Open-source visual builder on top of LangChain
- Drag-and-drop nodes to design AI pipelines
- Exports workflows as Python code → developer-friendly
- Best for prototyping and teaching LangChain apps

Flowise

- Open-source low/no-code editor for LangChain pipelines
- Build chatbots, RAG systems, assistants with minimal coding
- Deploy workflows directly as APIs
- Best for non-coders or teams needing quick deployment

Building a Visual Agent Workflow

Step- by-Step Process

- Start with an empty canvas in LangFlow/Flowise
- Drag in an LLM component (e.g., OpenAI GPT)
- Add a Tool node (Calculator, Web Search)
- Add an Agent node to coordinate
- Connect components with visual wires
- Configure node properties (API keys, parameters)
- Deploy and test with queries like "What is the population of Japan plus 100?"
- Visual builders handle the complex ReAct prompting, tool registration and agent logic behind the scenes!

Debugging & Understanding Agent Behaviour

1. Agent receives query
"What is the population of japan plus 100?“

2. Agent reasoning
"I need to find japan's population first, then add 100 to it.“

3. Tool selection
"I should use the search API to find population data."

4. Tool use
Calls search API with "population of japan“ Gets result: "125.7 million (2021)“

5. Second tool selection
"Now I need to add 100 to this number." Calls calculator with "125700000 + 100“

6. Final answer
"The population of japan plus 100 is 125,700,100."

Key Takeaways

Enhanced capabilities

Agents overcome LLM limits by delegating specialised tasks to purpose-built tools

Implementation options

Build agents programmatically with LangChain/AutoGen or visually with LangFlow/Flowise

Reasoning transparency

Verbose mode and logs allow to inspect agent decisions for debugging and refinement

Key Takeaways

Next Steps

- Experiment with diverse tools beyond calculators—web search, database lookups, code execution and API calls
- Chain tools together for multi-step reasoning
- Build your first production-ready agent with integrated capabilities

Q & A

Thank you